

IT Vedant

**Cloud-Based Linux Server Deployment with GitHub
Version Control and Automation**

Batch Code: DR501

Name: Saad Momin

Guide:

Prof. Akhilesh Dubey

Contents

1. Introduction

- 1.1 Importance of Cloud Computing in DevOps
- 1.2 Role of Linux Administration in DevOps
- 1.3 Overview of AWS Infrastructure
- 1.4 Web Server Deployment
- 1.5 Role of Git and GitHub in DevOps
- 1.6 Importance of Bash Automation
- 1.7 Linux Log Monitoring and Troubleshooting
- 1.8 Purpose of the Project

2. Overview

- 2.1 Title
- 2.2 Background
- 2.3 Business Scenario
- 2.4 Description
- 2.5 Objectives
- 2.6 Project Requirements
- 2.7 Project Workflow
- 2.8 AWS Architecture Flow
- 2.9 Deliverables
- 2.10 Expected Outcome

3. Objectives

- 3.1 To Configure AWS Cloud Infrastructure
- 3.2 To Configure an Ubuntu Linux Server
- 3.3 To Deploy a Website Using Apache Web Server
- 3.4 To Implement Git and GitHub Version Control
- 3.5 To Automate Administrative Tasks Using Bash Scripting
- 3.6 To Monitor Linux System Logs
- 3.7 To Develop Troubleshooting Skills
- 3.8 To Gain Practical Knowledge of DevOps Practices
- 3.9 To Improve Technical Documentation Skills
- 3.10 To Prepare for a DevOps Career

4. Scope of the Project

5. Technologies and Tools Used

- 5.1 Amazon Web Services (AWS)
- 5.2 Ubuntu Linux
- 5.3 Apache Web Server
- 5.4 Git
- 5.5 GitHub
- 5.6 Bash Shell Scripting
- 5.7 SSH (Secure Shell)
- 5.8 Curl
- 5.9 Vim
- 5.10 HTML

6. Implementation

- 6.1 Overview
- 6.2 AWS Infrastructure Setup
- 6.3 Ubuntu Server Configuration
- 6.4 Website Deployment
- 6.5 Git and GitHub Implementation
- 6.6 Bash Automation Scripts
- 6.7 Log Monitoring and Troubleshooting
- 6.8 Project Documentation
- 6.9 Project Workflow
- 6.10 Implementation Summary

7. Implementation Results

8. Challenges Faced

9. Learning Outcomes

10. Conclusion

11. References

Chapter 1

Introduction

Modern IT infrastructure demands rapid deployment, automation, scalability, and efficient system management. DevOps integrates software development and IT operations by leveraging automation, version control, continuous integration, and infrastructure management to streamline application deployment and maintenance. This project demonstrates the implementation of core DevOps practices using AWS EC2, Ubuntu Linux, Apache Web Server, Git & GitHub, and Bash scripting for cloud-based server deployment, website hosting, automation, and Linux log monitoring, providing hands-on experience with industry-standard DevOps tools and workflows.

1.1 Importance of Cloud Computing in DevOps

Cloud computing provides on-demand, scalable, and highly available infrastructure for modern DevOps workflows. It enables rapid provisioning of compute, storage, networking, and security resources without the need for physical hardware. In this project, Amazon Web Services (AWS) is used to provision a cloud-based Ubuntu EC2 instance, configure the required networking components, and deploy a Linux server environment for website hosting, automation, and version-controlled application management.

1.2 Role of Linux Administration in DevOps

Linux is the preferred operating system for cloud and DevOps environments due to its stability, security, and extensive command-line capabilities. It enables efficient system administration, package management, service configuration, process monitoring, and automation. In this project, an Ubuntu Linux server was deployed on an AWS EC2 instance to perform system configuration, software installation, web server management, and log monitoring using Linux commands.

1.3 Overview of AWS Infrastructure

Amazon Web Services (AWS) is a cloud computing platform that provides a wide range of infrastructure and application services. AWS enables users to create virtual servers, configure networks, manage security, and deploy applications without managing physical hardware.

The AWS infrastructure created in this project includes:

- Virtual Private Cloud (VPC)
- Public Subnet
- Internet Gateway
- Route Table
- Security Group
- Ubuntu EC2 Instance

This infrastructure provides a secure environment for hosting the project website and performing DevOps-related operations.

1.4 Web Server Deployment

A web server hosts and delivers web content over the internet using the HTTP protocol. In this project, the Apache HTTP Server was installed and configured on an Ubuntu EC2 instance to host a static website. The deployment was verified by accessing the application through the EC2 instance's public IP address.

1.5 Role of Git and GitHub in DevOps

Version control is an important part of DevOps that helps developers and teams track changes, manage code, and collaborate effectively.

Git is a distributed version control system used to manage source code and project files. GitHub is a cloud-based platform that provides online repositories for storing and managing Git projects.

In this project, Git and GitHub were used to:

- Create and manage the project repository
- Track project changes
- Perform commits
- Create and merge branches
- Store project documentation and scripts
- Maintain project version history

1.6 Importance of Bash Automation

Automation is a fundamental DevOps practice that improves operational efficiency by eliminating repetitive manual tasks. Bash scripting enables automated system administration, service management, monitoring, and backup operations. In this project, Bash scripts were developed for **server health monitoring, Apache service monitoring, and website backup creation**, demonstrating practical Linux automation techniques.

1.7 Linux Log Monitoring and Troubleshooting

Log monitoring helps administrators identify issues, analyze system behavior, and troubleshoot problems effectively.

In this project, Linux system logs were analyzed to prepare a monitoring report containing:

- Failed login attempts
- SSH login activity
- System errors

- Warning messages

This activity helped in understanding basic security monitoring and troubleshooting techniques.

1.8 Purpose of the Project

The purpose of this project is to bridge the gap between theoretical knowledge and practical implementation of DevOps concepts. It provides hands-on experience in managing cloud resources, working with Linux servers, implementing automation, and following professional DevOps workflows.

Chapter 2

Overview

2.1 Title

Cloud-Based Linux Server Deployment with GitHub Version Control and Automation

2.2 Background

With the adoption of cloud computing and DevOps, organizations require scalable, automated, and reliable deployment environments. This project demonstrates the practical implementation of DevOps fundamentals by provisioning cloud infrastructure on AWS, configuring an Ubuntu Linux server, deploying a web application using Apache, implementing Git & GitHub for version control, automating administrative tasks with Bash scripting, and performing Linux log monitoring. It provides hands-on experience with core DevOps tools and workflows used in modern infrastructure management.

2.3 Business Scenario

In this project, a Junior DevOps Engineer provisions a cloud-based infrastructure to deploy and manage an organization's internal web application. The implementation includes configuring AWS EC2 and Ubuntu Linux, deploying a website using Apache, managing source code with Git & GitHub, automating administrative tasks through Bash scripting, and monitoring Linux system logs. This workflow reflects the core responsibilities of a DevOps Engineer in a production-oriented environment.

2.4 Description

This project implements a complete DevOps workflow for deploying and managing a static web application on AWS Cloud. A secure cloud infrastructure was provisioned using VPC, Public Subnet, Internet Gateway, Route Table, Security Group, and an Ubuntu EC2 instance. The server was configured with Apache, Git, Curl, and Vim for web hosting and system administration. Git & GitHub were used for version control and project management, while Bash scripts automated server health monitoring, Apache service monitoring, and website backup operations. Additionally, Linux log monitoring was performed to analyze system events, authentication logs, and security-related activities.

2.5 Objectives

The main objectives of this project are:

- To understand the practical implementation of DevOps concepts.
- To configure and manage AWS cloud infrastructure.
- To create and configure an Ubuntu Linux server.
- To install and manage required Linux packages and services.
- To deploy a website using Apache Web Server.

- To understand Git and GitHub version control workflow.
- To create and execute Bash automation scripts.
- To monitor Linux system logs for security and troubleshooting.
- To develop problem-solving and troubleshooting skills.
- To document the complete DevOps implementation professionally.

2.6 Project Requirements

The project requires various cloud, operating system, and DevOps tools for successful implementation.

Hardware Requirement:

- Internet Connection
- AWS Cloud Account
- Ubuntu Server (EC2 Instance)
- Apache Web Server
- Git
- GitHub Repository
- Bash Shell
- SSH Client
- Text Editor (Nano/Vim)

AWS Services Used:

- Amazon EC2
- Amazon VPC
- Security Groups
- Internet Gateway
- Route Tables

2.7 Project Workflow

The complete project workflow follows these steps:

Step 1: AWS Infrastructure Setup



Step 2: EC2 Instance Configuration



Step 3: Linux Server Setup



Step 4: Website Deployment



Step 5: Version Control Implementation



Step 6: Automation



Step 7: Log Monitoring



Step 8: Project Documentation

2.8 AWS Architecture Flow

The architecture of this project follows a simple cloud deployment model:

Internet



AWS Cloud



Virtual Private Cloud (VPC)



Public Subnet



Security Group



Ubuntu EC2 Instance



Apache Web Server



Hosted Website

The AWS infrastructure provides a secure environment where the Linux server hosts the website and performs DevOps-related operations.

2.9 Deliverables

The final deliverables of this project include:

- AWS infrastructure configuration
- Ubuntu EC2 server setup
- Apache-hosted website
- GitHub repository
- Bash automation scripts
- Linux log monitoring report
- README.md documentation
- Project screenshots

2.10 Expected Outcome

After completing this project, the expected outcome is a fully functional cloud-based Linux server environment capable of hosting a website, managing source code, performing automated administrative tasks, and monitoring system activities.

The project provides practical understanding of how DevOps engineers combine cloud infrastructure, automation, and monitoring practices to maintain efficient and reliable application environments.

Chapter 3

Objectives

The primary objective of this project is to implement the fundamental concepts of DevOps by creating a cloud-based Linux server environment and performing essential system administration, automation, and version control tasks. The project provides practical exposure to industry-standard tools and technologies used by DevOps engineers in real-world environments.

The specific objectives of this project are as follows:

3.1 To Configure AWS Cloud Infrastructure

The first objective of this project is to create and configure a secure AWS cloud environment. This includes setting up a Virtual Private Cloud (VPC), Public Subnet, Internet Gateway, Route Table, Security Group, and launching an Ubuntu EC2 instance. This helps in understanding the basic components of AWS networking and cloud infrastructure.

3.2 To Configure an Ubuntu Linux Server

The project aims to develop practical knowledge of Linux system administration by configuring an Ubuntu server. This includes updating the operating system, upgrading installed packages, managing software installations, and verifying that the server is properly configured for application deployment.

3.3 To Deploy a Website Using Apache Web Server

Another objective is to deploy a static website on the Ubuntu EC2 instance using Apache Web Server. The website contains project-related information and demonstrates the basic process of hosting a web application on a cloud server.

3.4 To Implement Git and GitHub Version Control

This project aims to provide practical experience with Git and GitHub for version control. The repository is used to maintain project files, track changes, create branches, merge updates, and store documentation in a structured manner.

3.5 To Automate Administrative Tasks Using Bash Scripting

Automation is an essential part of DevOps. This project focuses on creating Bash scripts to automate routine administrative tasks such as monitoring server health, checking the Apache service status, and creating website backups. These scripts reduce manual effort and improve system efficiency.

3.6 To Monitor Linux System Logs

The project includes monitoring Linux system logs to understand server activities and identify security-related events. Log analysis helps in detecting failed login attempts, monitoring SSH access, identifying system errors, and reviewing warning messages.

3.7 To Develop Troubleshooting Skills

A key objective of this project is to develop troubleshooting skills by identifying and resolving common issues related to AWS networking, Linux permissions, Apache services, SSH connectivity, and Git operations. This enhances problem-solving abilities required in DevOps environments.

3.8 To Gain Practical Knowledge of DevOps Practices

The project provides hands-on experience with cloud computing, Linux administration, automation, web server deployment, monitoring, and version control. It helps in understanding how these technologies work together in a DevOps workflow.

3.9 To Improve Technical Documentation Skills

Another objective is to prepare professional project documentation, including a README file, log monitoring report, screenshots, and a detailed project report. Proper documentation is an important practice in software development and DevOps projects.

3.10 To Prepare for a DevOps Career

The final objective of this project is to build a strong foundation in DevOps by applying theoretical concepts in a practical environment. The knowledge and skills gained through this project prepare learners for entry-level roles such as Junior DevOps Engineer, Cloud Support Associate, or Linux Administrator.

CHAPTER 4

SCOPE OF THE PROJECT

This project demonstrates the end-to-end implementation of a DevOps workflow for deploying and managing a static web application on AWS Cloud. A secure cloud infrastructure was provisioned using Amazon VPC, Public Subnet, Internet Gateway, Route Table, Security Group, and an Ubuntu EC2 instance. The server environment was configured by installing essential packages including Apache HTTP Server, Git, Curl, and Vim. A static website was deployed and hosted using Apache, with accessibility verified through the EC2 public IP address. Git & GitHub were utilized for source code management, version control, and project documentation. Additionally, Bash scripts were developed to automate server health monitoring, Apache service monitoring, and website backup operations. Linux log monitoring was performed to analyze authentication events, system logs, errors, and warnings, ensuring effective system administration and troubleshooting.

Chapter 5

Technologies And Tools Used

This project was implemented using various cloud services, operating systems, software tools, and DevOps technologies. Each technology played an important role in the successful deployment, automation, monitoring, and management of the cloud-based Linux server environment.

5.1 Amazon Web Services (AWS)

Amazon Web Services (AWS) is a leading cloud computing platform that provides scalable, secure, and reliable cloud services. In this project, AWS was used to create the complete cloud infrastructure required for hosting the website and performing Linux administration tasks.

5.2 Ubuntu Linux

Ubuntu is an open-source Linux operating system widely used for cloud computing and server management. It provides a stable, secure, and command-line-based environment suitable for DevOps operations.

5.3 Apache Web Server

Apache HTTP Server is one of the most popular open-source web servers used to host websites and web applications.

In this project, Apache Web Server was installed on the Ubuntu server to deploy a static HTML website. The web server was configured to serve the webpage through the public IP address of the EC2 instance. The Apache service was also monitored using a Bash automation script.

5.4 Git

Git is a distributed version control system used to manage source code and track changes made during software development. Git ensured proper version management of all project files.

5.5 GitHub

GitHub is a cloud-based platform that hosts Git repositories and supports collaboration, version control, and project management.

The complete project, including documentation, Bash scripts, screenshots, and source files, was stored in a GitHub repository. GitHub provided secure online storage and version history for the project.

5.6 Bash Shell Scripting

Bash is a command-line shell and scripting language used for automating Linux administrative tasks.

Three Bash scripts were created in this project:

- Server Health Check Script
- Apache Service Monitor Script
- Website Backup Script

5.7 SSH (Secure Shell)

SSH is a secure network protocol used to remotely access Linux servers over an encrypted connection.

In this project, SSH was used to connect securely to the Ubuntu EC2 instance from the local computer using a private key (.pem file). This allowed remote server configuration and administration.

5.8 Curl

Curl is a command-line tool used for transferring data using various network protocols.

In this project, Curl was installed to verify network connectivity and test communication with web resources from the Ubuntu server.

5.9 Vim

Vim is a powerful text editor available in Linux operating systems.

It was used to edit configuration files, scripts, and other text-based files directly from the command line during server configuration.

5.10 HTML

HTML (HyperText Markup Language) is the standard markup language used to create web pages.

A simple HTML webpage was developed for this project containing:

- Project Title
- Student Name
- Batch Code
- Current Date
- Hosting Message

The webpage was deployed using Apache Web Server and successfully accessed through the EC2 public IP address.

Chapter 6

Implementation

6.1 Overview

The project was implemented using a structured DevOps workflow, starting with AWS cloud infrastructure provisioning and progressing through Ubuntu server configuration, Apache web server deployment, Git & GitHub version control, Bash automation, and Linux log monitoring. Each phase was executed systematically to ensure a secure, reliable, and fully functional cloud-based server environment, with comprehensive project documentation maintained throughout the implementation.

6.2 AWS Infrastructure Setup

The project began with provisioning the required infrastructure on Amazon Web Services (AWS). A secure network environment was established using a Virtual Private Cloud (VPC), Public Subnet, Internet Gateway, and Route Table. A Security Group was configured to allow SSH (Port 22) for remote administration and HTTP (Port 80) for web access. Finally, an Ubuntu EC2 instance was launched and securely accessed using SSH key-based authentication.

6.3 Ubuntu Server Configuration

Following the EC2 instance deployment, the Ubuntu Linux server was configured for application hosting. System packages were updated and upgraded, and essential tools including Git, Apache HTTP Server, Curl, and Vim were installed. The Apache service was verified and enabled to ensure proper web server functionality, preparing the server for website deployment and ongoing system administration.

6.4 Website Deployment

A static HTML webpage was developed and deployed on the Apache HTTP Server by placing the index.html file in the default web root directory (/var/www/html). The deployment was validated by accessing the application through the public IP address of the AWS EC2 instance, confirming successful web server configuration and application hosting.

6.5 Git and GitHub Implementation

Git was utilized for source code management and version control throughout the project. A local Git repository was initialized, project files were committed, and a remote GitHub repository (DevOps-Fundamentals-Project) was created for centralized storage. Standard version control operations, including commits, branch creation, merging, and remote synchronization, were performed to ensure efficient collaboration, change tracking, and secure project management.

6.6 Bash Automation Scripts

Three Bash scripts were developed to automate common system administration tasks.

Server Health Check Script: A script was created to monitor the overall health of the Linux server. It displays information such as CPU usage, memory usage, disk usage, logged-in users, running processes, and server uptime.

Apache Service Monitor Script: A monitoring script was developed to check the status of the Apache Web Server. If the service is found to be stopped, the script automatically restarts Apache and displays a confirmation message.

Website Backup Script: A backup script was created to compress the website files, generate a timestamp, and store the backup in a dedicated backup directory. This helps maintain copies of the website for recovery purposes.

6.7 Log Monitoring and Troubleshooting

Linux system log files were analyzed to monitor server activities and identify potential issues.

The log monitoring process included checking:

- Failed login attempts
- SSH login activity
- System error messages
- Warning messages

A structured log report was prepared and saved as log-report.txt. The report also included a troubleshooting summary describing common issues and their possible solutions.

Log monitoring provided valuable insight into server security and system performance.

6.8 Project Documentation

Proper documentation was maintained throughout the project.

The project repository included:

- README.md
- HTML website files
- Bash automation scripts
- Log monitoring report
- AWS and implementation screenshots

A detailed project report was also prepared to explain the implementation process, technologies used, challenges faced, and learning outcomes.

6.9 Project Workflow

The implementation followed the workflow shown below:

AWS Infrastructure Setup



Ubuntu Server Configuration



Apache Web Server Installation



Website Deployment



Git & GitHub Version Control



Bash Automation Scripts



Log Monitoring & Troubleshooting



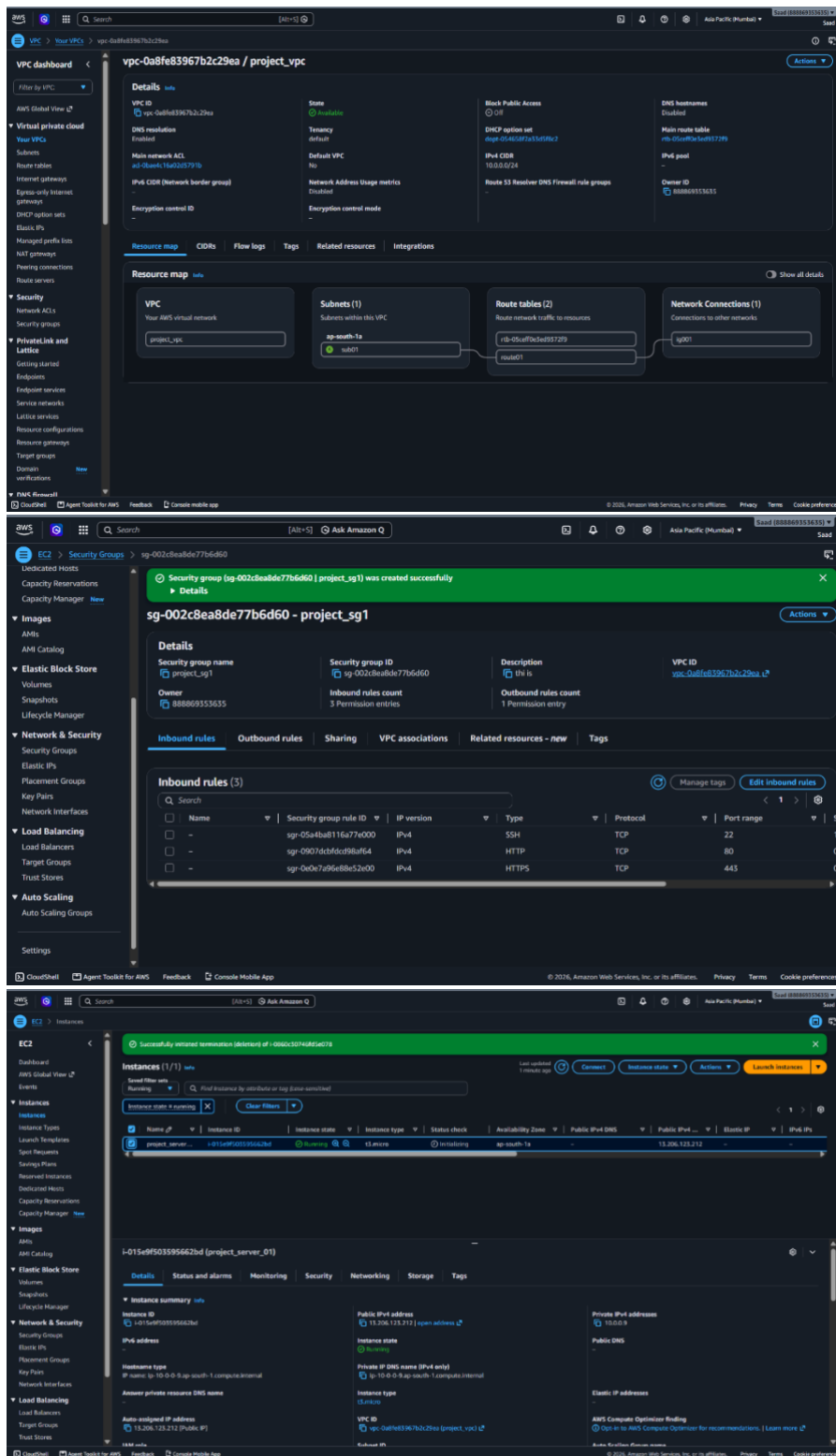
Project Documentation

6.10 Implementation Summary

The project successfully integrated AWS Cloud, Ubuntu Linux, Apache HTTP Server, Git & GitHub, Bash scripting, and Linux log monitoring into a unified DevOps workflow. A cloud-based web server was provisioned, configured, and deployed with automated administrative tasks and version-controlled project management. The implementation provided practical experience in cloud infrastructure provisioning, Linux system administration, deployment automation, version control, and system monitoring, strengthening the core technical competencies required in modern DevOps practices.

Chapter 7

Implementation Results



```

ubuntu@ip-10-0-0-9:~$
Saad@Aspire-Lite-15 MINGW64 ~/Downloads
$ ssh -i "demo222.pem" ubuntu@13.206.123.212
[1]~$ stopped ssh -i "demo222.pem" ubuntu@13.206.123.212
Saad@Aspire-Lite-15 MINGW64 ~/Downloads
$ ssh -i "demo222.pem" ubuntu@13.206.123.212
ssh: connect to host 13.206.123.212 port 22: Connection timed out
Saad@Aspire-Lite-15 MINGW64 ~/Downloads
$ ssh -i "demo222.pem" ubuntu@13.206.123.212
ssh: connect to host 13.206.123.212 port 22: Connection timed out
Saad@Aspire-Lite-15 MINGW64 ~/Downloads
$ K
Saad@Aspire-Lite-15 MINGW64 ~/Downloads
$ ssh -i "demo222.pem" ubuntu@13.206.123.212
The authenticity of host '13.206.123.212 (13.206.123.212)' can't be established.
ED25519 key fingerprint is: SHA256:z+qQ8DA8c7j9ugoSMK8esXJfPwLDvK85qc8H/5I
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '13.206.123.212' (ED25519) to the list of known hosts.
Welcome to Ubuntu 20.04 LTS (GNU/Linux 7.0.0-1006-aws x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sun Jul 19 11:22:26 UTC 2026

System load: 0.0          Temperature: ~273.1 C
Usage of /:  10.8% of 6.61GB    Processes: 117
Memory usage: 27%            Users logged in: 0
Swap usage:  0%              IPv4 address for ens5: 10.0.0.9

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/*-copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

ubuntu@ip-10-0-0-9:~$

```

```

ubuntu@ip-10-0-0-9:~$
Saad@Aspire-Lite-15 MINGW64 ~
$ cd /c/Users/Saad/Downloads
Saad@Aspire-Lite-15 MINGW64 ~/Downloads
$ ls
ASSIGN.pdf                                desktop.ini
Assignment.pdf                            'induction form(1).pdf'
Git-2.54.0-64-bit.exe*                   jenk.pem
ITVedant DevOps Fundamentals Mini Project (1).pdf*  jenkins_vpc.pem
Jenkins-Assignment.pdf                   proj.pem
Milhaan.png                             pycharm-2025.3.1.exe*
OneDriveSetup.exe*                       python-3.12.3-amd64.exe*
VSCodeOtherSetup-x64-1.116.0.exe*        python-manager-25.2.msix
WhatsApp Image 2026-01-03 at 7.50.50 PM (1).jpeg*  python-manager-26.0.msix
demo222.pem                             'transcreeen (1).exe'

Saad@Aspire-Lite-15 MINGW64 ~/Downloads
$ ssh -i "demo222.pem" ubuntu@13.201.123.42
The authenticity of host '13.201.123.42 (13.201.123.42)' can't be established.
ED25519 key fingerprint is: SHA256:z+qQ8DA8c7j9ugoSMK8esXJfPwLDvK85qc8H/5I
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:4: 13.206.123.212
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '13.201.123.42' (ED25519) to the list of known hosts.
Welcome to Ubuntu 20.04 LTS (GNU/Linux 7.0.0-1006-aws x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sun Jul 19 14:43:00 UTC 2026

System load: 0.0          Temperature: ~273.1 C
Usage of /:  11.1% of 6.61GB    Processes: 119
Memory usage: 26%            Users logged in: 0
Swap usage:  0%              IPv4 address for ens5: 10.0.0.9

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

Last login: Sun Jul 19 11:22:27 2026 from 106.211.105.202
ubuntu@ip-10-0-0-9:~$ sudo apt upgrade -y
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0
ubuntu@ip-10-0-0-9:~$

```

```

ubuntu@ip-10-0-0-9:~$ sudo apt upgrade -y
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0

```

```

ubuntu@ip-10-0-0-9:~$ sudo apt install git -y
git is already the newest version (1:2.53.0-1ubuntu1).
git set to manually installed.
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0
ubuntu@ip-10-0-0-9:~$ git --version
git version 2.53.0

```

```

ubuntu@ip-10-0-0-9:~$
ubuntu@ip-10-0-0-9:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-07-19 14:53:45 UTC; 42s ago
   Invocation: 8c2e4d9302a260dc1cfe9f2b2a9
     Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 2468 (apache2)
   Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0 0/sec"
     Tasks: 55 (limit: 627)
    Memory: 5.0M (peak: 3.8M)
      CPU: 97ms
   CGroup: /system.slice/apache2.service
           └─2468 /usr/sbin/apache2 -k start -DFOREGROUND
             └─2482 /usr/sbin/apache2 -k start -DFOREGROUND
               └─2483 /usr/sbin/apache2 -k start -DFOREGROUND

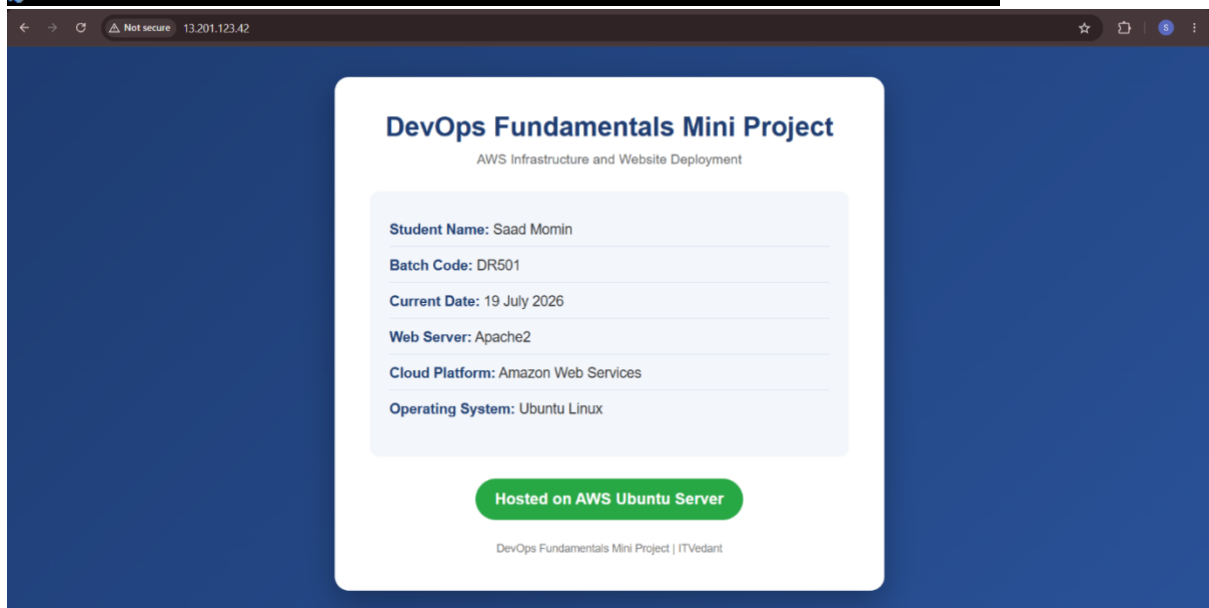
Jul 19 14:53:45 ip-10-0-0-9 systemd[1]: Starting apache2.service - The Apache HTTP Server...
Jul 19 14:53:45 ip-10-0-0-9 systemd[1]: 2468: HDROSS: apache2: Could not reliably determine the server's fully qualified domain name, using 10.0.0.9. Set the 'ServerName' directive globally to suppress this message.
Jul 19 14:53:45 ip-10-0-0-9 systemd[1]: Started apache2.service - The Apache HTTP Server.

ubuntu@ip-10-0-0-9:~$ curl --version
curl 8.18.0 (x86_64-pc-linux-gnu) libcurl/8.18.0 OpenSSL/3.3.5 zlib/1.3.1 brotli/1.2.0 zstd/1.5.7 libidn2/2.3.8 libpsl/0.21.2 libssh2/1.11.1 nghttp/2.1.68.0 librtmp/2.3 mit-krb5/1.22.1 OpenSSL/2.6.10
Release data: 2026-01-07, security patched: 8.18.0-1ubuntu2.3
Protocols: dict file ftps gopher gophers http https imap imaps ipfs ipns ldap ldaps mqtt pop3 pop3s rtmp rtsp scp sftp smb smbs smtp smtps telnet tftp ws wss
Features: alt-svc AsynchDNS brotli GSS-API HSTS HTTP2 HTTPS-proxy IDN IPv6 Kerberos Largefile libz NTLM SPNEGO SSL threadsafe TLS-SRP UnixSockets zstd
ubuntu@ip-10-0-0-9:~$

```

```
ubuntu@ip-10-0-0-9: ~  
ubuntu@ip-10-0-0-9:~$ vim --version  
VIM - Vi IMproved 9.1 (2024 Jan 02, compiled Jul 13 2026 17:06:53)  
Included patches: 1-948, 950-2141  
Modified by team+vim@tracker.debian.org  
Compiled by team+vim@tracker.debian.org  
ubuntu@ip-10-0-0-9:~$ cd /var/www/html  
ubuntu@ip-10-0-0-9:/var/www/html$ ls  
index.html  
ubuntu@ip-10-0-0-9:/var/www/html$
```

```
<!DOCTYPE html>  
<html>  
<head>  
  <title>DevOps Fundamentals Mini Project</title>  
</head>  
<body>  
  <h1>DevOps Fundamentals Mini Project</h1>  
  <p><strong>Student Name:</strong> Saad Momin</p>  
  <p><strong>Batch Code:</strong> DR501</p>  
  <p><strong>Date:</strong> 19 July 2026</p>  
  <h2>Hosted on AWS Ubuntu Server</h2>  
</body>  
</html>  
~  
~
```



```
ubuntu@ip-10-0-0-9:~$ mkdir DevOpsMiniProject  
ubuntu@ip-10-0-0-9:~$ cd DevOpsMiniProject  
ubuntu@ip-10-0-0-9:~/DevOpsMiniProject$ cp /var/www/html/index.html .  
ubuntu@ip-10-0-0-9:~/DevOpsMiniProject$ ls  
index.html  
ubuntu@ip-10-0-0-9:~/DevOpsMiniProject$
```

```

ubuntu@ip-10-0-0-9:~/DevOpsMiniProject$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: will change to "main" in Git 3.0. To configure the initial branch name
hint: to use in all of your new repositories, which will suppress this warning,
hint: call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
hint:
hint: Disable this message with "git config set advice.defaultBranchName false"
Initialized empty Git repository in /home/ubuntu/DevOpsMiniProject/.git/
ubuntu@ip-10-0-0-9:~/DevOpsMiniProject$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        index.html

nothing added to commit but untracked files present (use "git add" to track)
ubuntu@ip-10-0-0-9:~/DevOpsMiniProject$ 
ubuntu@ip-10-0-0-9:~/DevOpsMiniProject$ git checkout master
Switched to branch 'master'
Your branch is ahead of 'origin/master' by 1 commit.
  (use "git push" to publish your local commits)
ubuntu@ip-10-0-0-9:~/DevOpsMiniProject$ git merge dev
Updating 27b8000..1fd7b95
Fast-forward
 README.md | 1 +
 1 file changed, 1 insertion(+)
ubuntu@ip-10-0-0-9:~/DevOpsMiniProject$ git remote set-url origin https://github.com/saad7804/DevOps-Fundamentals-Project.git
ubuntu@ip-10-0-0-9:~/DevOpsMiniProject$ git remote -v
origin  https://github.com/saad7804/DevOps-Fundamentals-Project.git (fetch)
origin  https://github.com/saad7804/DevOps-Fundamentals-Project.git (push)
ubuntu@ip-10-0-0-9:~/DevOpsMiniProject$ vim README.md
ubuntu@ip-10-0-0-9:~/DevOpsMiniProject$ git add .
ubuntu@ip-10-0-0-9:~/DevOpsMiniProject$ git commit -m "Completed DevOps Fundamentals Project"
[master 27b8000] Completed DevOps Fundamentals Project
 7 files changed, 170 insertions(+)
 create mode 100644 Backup/website_backup_2026-07-19_16-04-01.tar.gz
 create mode 100644 README.md
 create mode 100644 log-report.txt
 create mode 100755 scripts/apache-monitor.sh
 create mode 100755 scripts/backup.sh
 create mode 100755 scripts/health-check.sh
 create mode 100755 scripts/system_info.sh
ubuntu@ip-10-0-0-9:~/DevOpsMiniProject$ git checkout -b dev
Switched to a new branch 'dev'
ubuntu@ip-10-0-0-9:~/DevOpsMiniProject$ echo "Project completed successfully." >> README.md
ubuntu@ip-10-0-0-9:~/DevOpsMiniProject$ git add README.md
git commit -m "Updated README in dev branch"
[dev 1fd7b95] Updated README in dev branch
 1 file changed, 1 insertion(+)

```

```
ubuntu@ip-10-0-0-9:~/DevOpsMiniProject$ git push origin master
Username for 'https://github.com': saad7804
Password for 'https://saad7804@github.com':
Enumerating objects: 15, done.
Counting objects: 100% (15/15), done.
Delta compression using up to 2 threads
Compressing objects: 100% (14/14), done.
Writing objects: 100% (14/14), 3.94 KiB | 1.97 MiB/s, done.
Total 14 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), done.
To https://github.com/saad7804/DevOps-Fundamentals-Project.git
   643c16e..1fd7b95  master -> master
ubuntu@ip-10-0-0-9:~/DevOpsMiniProject$ git status
On branch master
Your branch is up to date with 'origin/master'.
```

nothing to commit, working tree clean

```
ubuntu@ip-10-0-0-9:~/DevOpsMiniProject$ vim README.md
ubuntu@ip-10-0-0-9:~/DevOpsMiniProject$ git add README.md
git commit -m "Updated README with project documentation"
git push origin master
[master e45968b] Updated README with project documentation
 1 file changed, 86 insertions(+), 21 deletions(-)
Username for 'https://github.com': saad7804
Password for 'https://saad7804@github.com':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 1.15 KiB | 1.15 MiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/saad7804/DevOps-Fundamentals-Project.git
   1fd7b95..e45968b  master -> master
```



```

ubuntu@ip-10-0-0-9:~/DevOpsMiniProject/scripts$ vim health-check.sh
ubuntu@ip-10-0-0-9:~/DevOpsMiniProject/scripts$ chmod +x health-check.sh
ubuntu@ip-10-0-0-9:~/DevOpsMiniProject/scripts$ ./health-check.sh
=====
SERVER HEALTH CHECK
=====

CPU Usage:
%Cpu(s):  0.0 us,   0.0 sy,   0.0 ni,100.0 id,   0.0 wa,   0.0 hi,   0.0 si,   0.0 st

Memory Usage:
          total        used        free      shared  buff/cache   available
Mem:      908Mi       355Mi       202Mi        3.1Mi       481Mi       552Mi
Swap:      0B           0B           0B

Disk Usage:
Filesystem      Size  Used Avail Use% Mounted on
/dev/root        6.7G  2.7G  4.0G  41% /
tmpfs            455M   0  455M   0% /dev/shm
tmpfs            182M  928K  181M   1% /run
efivarfs         128K  3.3K  120K   3% /sys/firmware/efi/efivars
tmpfs            455M   0  455M   0% /tmp
/dev/nvme0nlp13  989M  163M  760M  18% /boot
/dev/nvme0nlp15  105M   6.3M   99M   7% /boot/efi
none             1.0M   0   1.0M   0% /run/credentials/serial-getty@ttyS0.service
none             1.0M   0   1.0M   0% /run/credentials/getty@tty1.service
tmpfs            91M   8.0K   91M   1% /run/user/1000
none             1.0M   0   1.0M   0% /run/credentials/systemd-resolved.service
none             1.0M   0   1.0M   0% /run/credentials/systemd-journald.service
none             1.0M   0   1.0M   0% /run/credentials/systemd-networkd.service

Logged-in Users:

Running Processes:
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.1  1.7  24732 15996 ?        Ss   14:39   0:08 /usr/lib/systemd/systemd --system --deserialize=39
root    20821  0.0  4.7 448156 43972 ?        Ssl  15:45   0:00 /usr/libexec/fwupd/fwupd
root       604  0.0  2.7 1959600 25520 ?        Ssl  14:39   0:01 /usr/lib/snapd/snapd
root     1501  0.0  1.9 1832820 17776 ?        Ssl  14:44   0:01 /snap/amazon-ssm-agent/13349/amazon-ssm-agent
message+  584  0.0  0.5   9104  5428 ?        Ss   14:39   0:01 @dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation --syslog-only
root    3755  0.0  0.0      0     0 ?        I    15:03   0:00 [kworker/u8:0-events_unbound]
root       56  0.0  0.0      0     0 ?        S    14:39   0:00 [kswapd0]
root       34  0.0  0.0      0     0 ?        I    14:39   0:00 [kworker/u8:1-events_unbound]
root    3262  0.0  0.0      0     0 ?        I    15:03   0:00 [kworker/0:0-events]

Server Uptime:
15:57:30 up 1:18, 2 users, load average: 0.01, 0.01, 0.00

=====
Health check completed successfully.
=====

ubuntu@ip-10-0-0-9:~/DevOpsMiniProject/scripts$ vim apache-monitor.sh
ubuntu@ip-10-0-0-9:~/DevOpsMiniProject/scripts$ chmod +x apache-monitor.sh
ubuntu@ip-10-0-0-9:~/DevOpsMiniProject/scripts$ ./apache-monitor.sh
=====
APACHE SERVICE MONITOR
=====

Apache service is active and running.
=====

ubuntu@ip-10-0-0-9:~/DevOpsMiniProject/scripts$ vim backup.sh
ubuntu@ip-10-0-0-9:~/DevOpsMiniProject/scripts$ chmod +x backup.sh
ubuntu@ip-10-0-0-9:~/DevOpsMiniProject/scripts$ ./backup.sh
=====
WEBSITE BACKUP SCRIPT
=====

Website backup created successfully.
Backup file: /home/ubuntu/DevOpsMiniProject/Backup/website_backup_2026-07-19_16-04-01.tar.gz

Available backups:
total 4.0K
-rw-rw-r-- 1 ubuntu ubuntu 1.1K Jul 19 16:04 website_backup_2026-07-19_16-04-01.tar.gz
=====

```

```

Linux Log Monitoring Report
=====

1. Failed Login Attempts
-----
Jul 19 16:12:15 ip-10-0-0-9 sudo[2121]: ubuntu : TTY=/dev/pts/1 : PWD=/home/ubuntu/DevOpsMiniProject : USER=root : COMMAND=/usr/bin/grep -i failed password /var/log/auth.log

2. SSH Login Activity
-----
Jul 19 15:40:52 ip-10-0-0-9 sshd-session[20787]: error: maximum authentication attempts exceeded for ubuntu from 112.185.16.157 port 60244 ssh2 [preauth]
Jul 19 15:40:52 ip-10-0-0-9 sshd-session[20787]: Disconnecting authenticating user ubuntu 112.185.16.157 port 60244: Too many authentication failures [preauth]
Jul 19 15:41:08 ip-10-0-0-9 sshd-session[20798]: Received disconnect from 112.185.16.157 port 60710:11: disconnected by user [preauth]
Jul 19 15:41:08 ip-10-0-0-9 sshd-session[20798]: Disconnecting from authenticating user ubuntu 112.185.16.157 port 60710 [preauth]
Jul 19 15:41:17 ip-10-0-0-9 sshd-session[20800]: Invalid user pi from 112.185.16.157 port 32824
Jul 19 15:41:22 ip-10-0-0-9 sshd-session[20800]: Received disconnect from 112.185.16.157 port 32824:11: disconnected by user [preauth]
Jul 19 15:41:22 ip-10-0-0-9 sshd-session[20800]: Disconnecting from invalid user pi 112.185.16.157 port 32824 [preauth]
Jul 19 15:41:30 ip-10-0-0-9 sshd-session[20810]: Invalid user baikal from 112.185.16.157 port 33182
Jul 19 15:41:31 ip-10-0-0-9 sshd-session[20810]: Received disconnect from 112.185.16.157 port 33182:11: disconnected by user [preauth]
Jul 19 15:41:31 ip-10-0-0-9 sshd-session[20810]: Disconnecting from invalid user baikal 112.185.16.157 port 33182 [preauth]

3. System Errors
-----
Jul 19 15:37:45 ip-10-0-0-9 sshd-session[20750]: error: maximum authentication attempts exceeded for invalid user user from 112.185.16.157 port 55452 ssh2 [preauth]
Jul 19 15:37:59 ip-10-0-0-9 sshd-session[20752]: error: maximum authentication attempts exceeded for invalid user user from 112.185.16.157 port 55834 ssh2 [preauth]
Jul 19 15:38:27 ip-10-0-0-9 sshd-session[20761]: error: maximum authentication attempts exceeded for invalid user ftpuser from 112.185.16.157 port 56524 ssh2 [preauth]
Jul 19 15:38:43 ip-10-0-0-9 sshd-session[20763]: error: maximum authentication attempts exceeded for invalid user ftpuser from 112.185.16.157 port 56914 ssh2 [preauth]
Jul 19 15:39:13 ip-10-0-0-9 sshd-session[20767]: error: maximum authentication attempts exceeded for invalid user test1 from 112.185.16.157 port 57704 ssh2 [preauth]
Jul 19 15:39:27 ip-10-0-0-9 sshd-session[20769]: error: maximum authentication attempts exceeded for invalid user test1 from 112.185.16.157 port 58092 ssh2 [preauth]
Jul 19 15:39:54 ip-10-0-0-9 sshd-session[20773]: error: maximum authentication attempts exceeded for invalid user test2 from 112.185.16.157 port 58718 ssh2 [preauth]
Jul 19 15:40:08 ip-10-0-0-9 sshd-session[20775]: error: maximum authentication attempts exceeded for invalid user test2 from 112.185.16.157 port 59132 ssh2 [preauth]
Jul 19 15:40:35 ip-10-0-0-9 sshd-session[20785]: error: maximum authentication attempts exceeded for ubuntu from 112.185.16.157 port 59816 ssh2 [preauth]
Jul 19 15:40:52 ip-10-0-0-9 sshd-session[20787]: error: maximum authentication attempts exceeded for ubuntu from 112.185.16.157 port 60244 ssh2 [preauth]

4. Warning Messages
-----
Jul 19 15:37:59 ip-10-0-0-9 sshd-session[20752]: error: maximum authentication attempts exceeded for invalid user user from 112.185.16.157 port 55834 ssh2 [preauth]
Jul 19 15:38:27 ip-10-0-0-9 sshd-session[20761]: error: maximum authentication attempts exceeded for invalid user ftpuser from 112.185.16.157 port 56524 ssh2 [preauth]
Jul 19 15:38:43 ip-10-0-0-9 sshd-session[20763]: error: maximum authentication attempts exceeded for invalid user ftpuser from 112.185.16.157 port 56914 ssh2 [preauth]
Jul 19 15:39:13 ip-10-0-0-9 sshd-session[20767]: error: maximum authentication attempts exceeded for invalid user test1 from 112.185.16.157 port 57704 ssh2 [preauth]
Jul 19 15:39:27 ip-10-0-0-9 sshd-session[20769]: error: maximum authentication attempts exceeded for invalid user test1 from 112.185.16.157 port 58092 ssh2 [preauth]
Jul 19 15:39:54 ip-10-0-0-9 sshd-session[20773]: error: maximum authentication attempts exceeded for invalid user test2 from 112.185.16.157 port 58718 ssh2 [preauth]
Jul 19 15:40:08 ip-10-0-0-9 sshd-session[20775]: error: maximum authentication attempts exceeded for invalid user test2 from 112.185.16.157 port 59132 ssh2 [preauth]
Jul 19 15:40:35 ip-10-0-0-9 sshd-session[20785]: error: maximum authentication attempts exceeded for ubuntu from 112.185.16.157 port 59816 ssh2 [preauth]
Jul 19 15:40:52 ip-10-0-0-9 sshd-session[20787]: error: maximum authentication attempts exceeded for ubuntu from 112.185.16.157 port 60244 ssh2 [preauth]
Jul 19 15:41:09 ip-10-0-0-9 kernel: ena 0000:00:05:01 Type was not set for devlink port.

Report generated on: Sun Jul 19 16:22:21 UTC 2026

```

Chapter 8

Challenges Faced

During the implementation of the Cloud-Based Linux Server Deployment with GitHub Version Control and Automation project, several practical challenges were encountered. These challenges provided valuable learning opportunities and helped in developing problemsolving, troubleshooting, and technical skills. Each issue was resolved by analyzing the root cause and applying the appropriate solution.

8.1 AWS Infrastructure Configuration

The initial challenge involved correctly configuring the AWS networking infrastructure, including the VPC, Public Subnet, Internet Gateway, Route Table, and Security Group. Proper network configuration and resource associations were essential to provide secure internet connectivity for the Ubuntu EC2 instance. After validating the networking setup, the instance was successfully provisioned and made accessible.

8.2 SSH Connection to EC2 Instance

Establishing a secure SSH connection to the Ubuntu EC2 instance required correct key-based authentication, appropriate private key permissions, and Security Group configuration to allow SSH (Port 22) access. Successfully configuring these components enabled secure remote administration and reinforced the principles of secure server access in cloud environments.

8.3 Apache Web Server Configuration

Following the installation of the Apache HTTP Server, the service was verified using `systemctl` to ensure proper operation. The static website was then deployed to the default web root directory and successfully accessed through the EC2 public IP address, validating the web server configuration and deployment process.

8.4 Website Deployment

Deploying the static website involved placing the HTML file in the Apache document root (`/var/www/html`) and verifying the deployment. Successful access through the EC2 public IP address confirmed correct web server configuration and demonstrated the complete workflow of hosting a static website on a Linux-based Apache server.

8.5 Git and GitHub Operations

Managing the project using Git and GitHub required performing various version control operations such as repository initialization, staging files, committing changes, creating branches, merging branches, and pushing the project to GitHub.

While working on the project, proper repository structure and file management were maintained to ensure successful synchronization between the local repository and GitHub.

This improved understanding of version control and collaborative project management.

8.6 Bash Script Development

Writing Bash scripts required understanding Linux commands, shell scripting syntax, variables, conditional statements, and file permissions.

Each script was tested individually to verify that it executed successfully and performed the required automation tasks, including server health monitoring, Apache service monitoring, and website backup creation.

This activity strengthened Linux automation skills.

8.7 Log Monitoring and Troubleshooting

Analyzing Linux system logs was a new learning experience.

The challenge was identifying the appropriate log information related to authentication, SSH login activity, system errors, and warning messages.

After analyzing the logs, a structured log monitoring report was prepared and saved as logreport.txt, improving understanding of Linux system monitoring and troubleshooting.

8.8 Overall Learning from the Challenges

Every challenge encountered during the project contributed to practical learning and skill development. Resolving issues related to AWS infrastructure, Linux server configuration, Apache Web Server, Git and GitHub, Bash scripting, and log monitoring improved technical knowledge and problem-solving abilities.

These experiences provided valuable hands-on exposure to real-world DevOps tasks and increased confidence in managing cloud-based Linux environments.

Chapter 9

Learning Outcomes

The successful completion of this project provided practical experience in implementing core DevOps concepts using AWS, Ubuntu Linux, Git & GitHub, Apache HTTP Server, Bash scripting, and Linux log monitoring. The key learning outcomes are as follows:

10.1 Linux Administration

Gained hands-on experience in Linux system administration, package management, file operations, permissions, and service management using the command line.

10.2 AWS Cloud Infrastructure

Learned to provision and configure AWS resources, including VPC, EC2, Subnets, Security Groups, and networking components.

10.3 EC2 Instance Management

Configured and managed an Ubuntu EC2 instance, established secure SSH access, and administered server services.

10.4 Web Server Deployment

Installed and configured the Apache HTTP Server to deploy and host a static website on an AWS EC2 instance.

10.5 Git & GitHub

Applied Git and GitHub for version control, including commits, branching, merging, and remote repository management.

10.6 Bash Scripting

Developed Bash scripts to automate server health monitoring, Apache service monitoring, and website backup tasks.

10.7 Log Monitoring

Analyzed Linux system and authentication logs to monitor server activities and perform basic troubleshooting.

10.8 Problem Solving

Resolved issues related to AWS networking, SSH connectivity, Apache configuration, and Git operations during implementation.

Chapter 10

Conclusion

The Cloud-Based Linux Server Deployment with GitHub Version Control and Automation project provided hands-on experience in core DevOps practices, including AWS cloud provisioning, Ubuntu Linux administration, Apache web server deployment, Git & GitHub version control, Bash scripting, and Linux log monitoring. The project involved configuring AWS infrastructure, deploying a static website, automating administrative tasks, and monitoring system logs. Practical challenges related to networking, SSH connectivity, and service configuration were resolved through troubleshooting, strengthening technical and problem-solving skills. Overall, the project established a strong foundation in DevOps workflows and industry-standard tools.

References

1. Amazon Web Services (AWS) Official Documentation <https://docs.aws.amazon.com>
2. Amazon EC2 Documentation <https://docs.aws.amazon.com/ec2/>
3. Amazon VPC Documentation <https://docs.aws.amazon.com/vpc/>
4. Ubuntu Official Documentation <https://ubuntu.com/server/docs>
5. Apache HTTP Server Official Website <https://httpd.apache.org/docs/>
6. Git Official Documentation <https://git-scm.com/doc>
7. GitHub Documentation <https://docs.github.com>
8. Linux Documentation Project <https://tldp.org>
9. GNU Bash Documentation <https://www.gnu.org/software/bash/manual/>
10. IT Vedant Official Website <https://www.itvedant.com>